

3.1 AI Solution Specification

Model: **Claude** qua **Claude Code CLI** (@anthropic-ai/claude-code), chạy headless như một subprocess do backend spawn. Không dùng chat interface, không tự dựng agent loop.

AI Use Cases

#	Use case	Ở đâu	Vì sao không deterministic được
U1	Đọc source code và rút ra knowledge base — stack, kiến trúc, route, selector, page object, fixture, auth flow	Hub (project-bootstrap)	Static analysis cho được AST, không cho được ngữ nghĩa "đây là màn hình login"
U2	Phân tích requirement — bóc AC, tìm chỗ mơ hồ, suy ra risk	Q-Agent (requirement-analyst)	Requirement là ngôn ngữ tự nhiên, không có schema
U3	Sinh test case có truy vết tới AC	Q-Agent (test-case-generator, test-case-reviewer)	Phủ AC là bài toán suy luận, không phải template
U4	Drive browser thật để xác minh từng step và ghi selector ổn định	Q-Agent (live-authoring + browser-harness)	Chọn selector ổn định nhất cần hiểu ngữ cảnh UI, không phải quy tắc cứng
U5	Sinh page object và spec Playwright	Q-Agent (automation-planner, page-object-author, automation-generator, automation-reviewer)	Code sinh ra phải khớp cấu trúc project đang có
U6	Phân loại failure và heal có mục tiêu	Q-Agent (execution-analyzer, page-object-healer)	Đọc log + DOM để phân biệt test sai với product sai
U7	Annotate screenshot, sinh comment ticket, sinh report	Q-Agent (screenshot-annotator, ticket-comment-generator, report-generator)	Viết cho người đọc

Cố tình không dùng AI: CRUD, encrypt/decrypt, ký và validate JWT, phân quyền, gọi Azure DevOps API, tính usage. Những chỗ đó có lời giải deterministic và phải deterministic.

AI Capabilities

Capability	Dùng ở
Đọc và suy luận trên codebase lớn (file tool, grep, glob)	U1, U5
Tool use — gọi CLI, đọc/ghi file, chạy lệnh	U1, U4, U5, U6
Browser control qua browser-harness (accessibility tree, dispatch event trên selector)	U4
Structured output theo JSON schema do backend pin	U2, U3, U6
Streaming — log, tiến độ, token, cost đẩy thẳng về browser	mọi use case

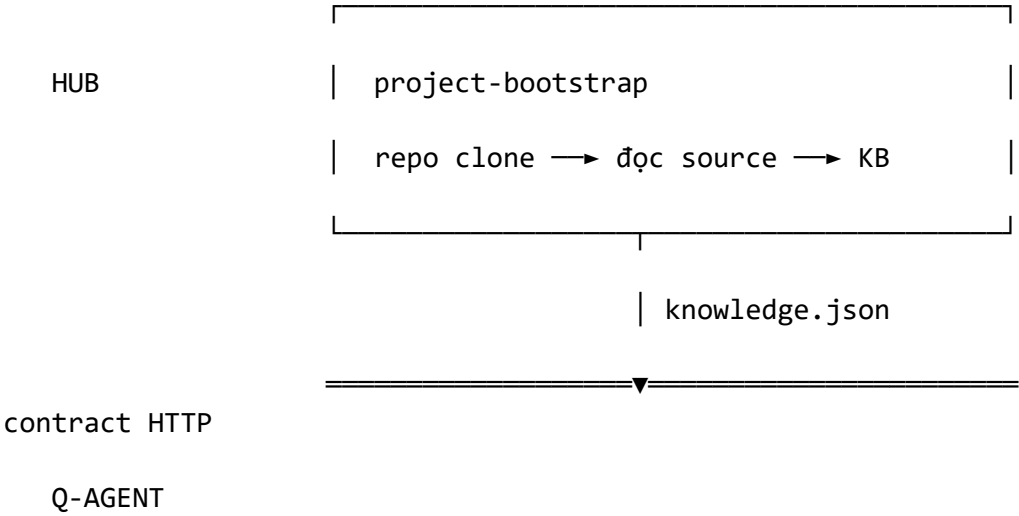
MCP (Azure DevOps)	U3 khi cần đọc thêm work item liên quan
--------------------	---

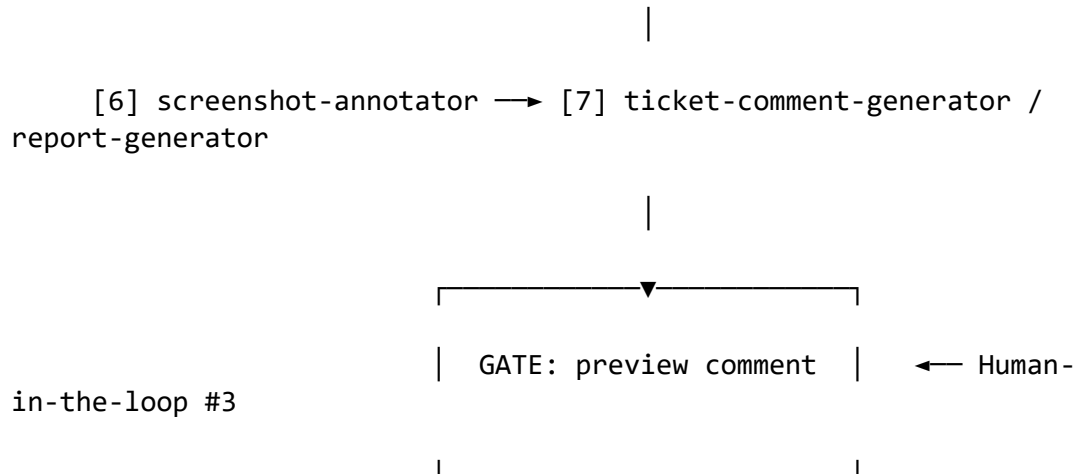
Input / Output

Bước	Input	Output
project-bootstrap	Repo đã clone (source thật), project config	knowledge.md (cho người) + knowledge.json (cho AI): stack, kiến trúc, domain, route, selector, auth flow, environment, page object và fixture tái dùng được. Kèm confidence score
requirement-analyst	Ticket đã normalise (description, AC, comment) + knowledge.json	Danh sách AC đã bóc tách, điểm mờ hồ, risk area
test-case-generator	Output trên + knowledge base	Test case theo format Azure DevOps, mỗi case map tới AC cụ thể. Trạng thái khởi tạo pending
live-authoring	Test case đã approved + base URL + test account + browser đã authenticated	Trace của từng step đã thực thi thật, kèm selector đã verify
automation-generator	Trace trên + page object hiện có	Spec Playwright + TypeScript, page object và fixture mới nếu cần
execution-analyzer	Log chạy + DOM lúc fail + spec	Failure class + đề xuất fix có mục tiêu
ticket-comment-generator	Kết quả run + evidence	Comment đã format, preview trước khi gửi

Ràng buộc quan trọng: prompt của caller **pin chính xác shape JSON** mà backend parse. Skill định nghĩa methodology và quality rule; caller định nghĩa contract dữ liệu. Hai thứ tách nhau.

AI Workflow





Vòng ghi ngược ở bước 5 là điểm đáng chú ý: **KB giàu lên theo thời gian**. Selector verify được trên app đang chạy được stamp timestamp kèm strategy đã hoạt động, và entry verified-at-runtime **thăng** entry suy ra từ source — merge sau không ghi đè nó.

Agent Workflow

Mỗi bước là một lần spawn Claude Code CLI, không phải một lần gọi Messages API. Điều đó cho:

Đặc điểm	Hệ quả
Agent có tool use thật (file, shell, browser)	Nó chạy được thay vì chỉ mô tả
Skill được inject làm system prompt cho đúng action đó	Không phải nhồi mọi thứ vào một prompt khổng lồ
Mỗi run có worktree/workspace riêng theo owner	Chạy song song không đụng nhau
Streaming stdout	Log, tiến độ, token, cost lên UI theo thời gian thực

browser-harness là điểm khác biệt lớn nhất so với "LLM sinh code": agent gắn vào một Chrome đã authenticated và:

- Thực thi từng step của test case **trên app đang chạy**. Không mô phỏng.
- Resolve element qua **accessibility tree**, ghi selector theo thứ tự ưu tiên cố định: data-testid → ARIA role + accessible name → label → CSS ổn định. Không :nth-child, không class trần, không structural combinator.
- Verify selector chứ không verify tọa độ**. click_at_xy trúng bất kỳ element nào ở điểm đó — thường là một <a> bên trong — trong khi spec sẽ click **selector đã ghi**, mà tâm của nó có thể là container không handle click. Nên mỗi interaction được dispatch trực tiếp trên chính selector sắp emit, rồi verify hiệu ứng thật (URL có đổi không).
- Tạo test data nếu thiếu**, qua UI, và bake setup đó vào spec để lần chạy sau tự đứng được.
- Emit spec** từ đúng những gì đã chạy.

Context / Prompt Strategy

Ba tầng, tách bạch:

Tầng	Nội dung	Ai sở hữu
Skill (SKILL.md)	Methodology, quality rule, output template. Backend inject làm system prompt cho đúng action đó	Đi kèm sản phẩm — 14 skill trong Q-Agent, 1 trong hub
Grounding	knowledge.json của chính repository: route thật, selector thật, page object, auth flow, environment	Sinh ra từ source, không phải người viết
Task	Ticket đã normalise + JSON schema mà backend sẽ parse	Caller pin cứng

Hệ quả trực tiếp: **người dùng không phải viết prompt**. Cùng một ticket thì ai chạy cũng ra kết quả tương đương. Và skill là **tài sản chung được review và cải tiến** — thấy test case bỏ sót một loại case thì sửa test-case-generator một lần, cả nhóm được hưởng.

Quản lý context: không nhồi cả repo vào prompt. knowledge.json là bản chứng cất đã có cấu trúc; agent đọc thêm file cụ thể qua tool khi cần. Đây là lý do build KB một lần rẻ hơn cho mọi run về sau.

Tool Usage

Tool	Dùng để	Ràng buộc
File read/write/glob/grep	Đọc source, viết spec và page object	Giới hạn trong workspace của owner
Shell	git clone, npx playwright test, npm	Không có chromium trong image api — build KB không cần browser
browser-harness CLI	Live-authoring, exploration	Chỉ chạy phía có browser (Local Agent hoặc host có Chrome)
MCP Azure DevOps	Đọc work item liên quan	PAT do hub cấp qua GET /connections/{id}/secret (agent audience)

Human-in-the-Loop

Ba gate, đều do người giữ, đều chặn thật (không phải cảnh báo):

#Gate	Chặn gì
1Review Center	Case sinh ra ở pending. Chỉ case approved đi tiếp sang Link và Automation. Reviewer đổi được automation type; case Manual không bao giờ đem sinh spec
2Create & Link có local mode	Dry-run: ghi local, không write lên provider
3Comment preview	Comment trả về ticket được preview trước, kèm status mapping cấu hình được — không phải một lần write thẳng

Nguyên tắc: **AI làm phần lặp, người làm phần phán đoán**. Ba gate nằm đúng ở ba chỗ có hậu quả ra ngoài hệ thống (dữ liệu trên Azure DevOps, kết luận về chất lượng sản phẩm).

Guardrails

Guardrail	Cơ chế
Placeholder gate	Pre-flight check trên spec vừa sinh: selector bịa, TODO stub, URL placeholder → verdict passed / blocked / rejected. Spec blocked surface lên UI là cần grounding, thay vì fail lặng lẽ lúc execution và bị đọc nhầm thành product defect
Bounded run	Cost ceiling + turn cap + wall-clock timeout cho mọi mode
Heal bounded	heal_max_attempts = 3, Playwright timeout rút ngắn, chạy trên model rẻ
Failure classification	test_defect / product_defect / flaky / environment / timeout — giữ cho một spec sai không bị đọc thành product bug
Selector policy	Cấm :nth-child, class trần, structural combinator ở tầng skill
Credential không rời hub	Provider PAT proxy qua hub; Claude credential ra qua một hàm, và grant chỉ reach được 3 route
Không fail-open	Không có cấu hình nào khiến "authentication không khả dụng" dẫn tới "cho vào"
Audit	Mọi thao tác đáng ghi vào audit log append-only, kèm run code
Usage attribution	Token và cost ghi theo owner mỗi lần gọi

AI Failure Scenarios

Scenario	Biểu hiện	Hệ thống xử lý
Model bịa selector	Spec chứa selector không tồn tại	Placeholder gate chặn trước execution → blocked
Model sinh test case thiếu coverage	AC không có case nào cover	Cột mapping AC → case hiển thị chỗ hở; reviewer thấy và bổ sung
Model sinh test case sai nghiệp vụ	Case nhìn hợp lý nhưng sai luồng	Gate 1 — reviewer từ chối
Knowledge base sai vì source đã đổi	Selector cũ	Trạng thái stale; entry verified-at-runtime thẳng entry suy ra từ source
Heal loop không hội tụ	Sửa 3 lần vẫn đỏ	Dừng ở 3 lần, trả failure class để người đọc
Model đọc product_defect thành test_defect	Heal một spec đúng cho tới khi nó "xanh" giả	Heal chỉ chạy khi có grounding DOM; failure class hiển thị để người phản bác
Run vượt ngân sách	Chi phí tăng không kiểm soát	Cost ceiling dừng run, usage hiển thị theo run
Claude credential hết hạn giữa run	Call bị reject	Status refreshable vs expired; refresh token ghi ngược về hub
Model không đi hết được live-harness	Không emit spec	Đây là hành vi đúng — không có spec còn hơn spec sai

Why AI Is Necessary

Ba phát biểu kiểm chứng được:

1. **Không có schema cho requirement.** AC viết bằng tiếng Việt/tiếng Anh tự do. Bất kỳ parser deterministic nào cũng chỉ phủ được tập con mà tác giả parser tưởng tượng ra.
2. **Không có ánh xạ cơ học từ source sang "màn hình login".** Rút ra route, selector, page object dùng lại được từ một codebase lạ đòi hỏi suy luận ngữ nghĩa. AST cho biết có một `<input>`; không cho biết nó là ô mật khẩu của luồng đăng nhập chính.
3. **Phân biệt "test sai" với "product sai" từ một log đỏ là bài toán suy luận.** Đây là chỗ tốn thời gian nhất của QA hôm nay, và cũng là chỗ automation truyền thống bỏ trống hoàn toàn.

Và một phát biểu ngược lại, cũng quan trọng: **phần còn lại của hệ thống cố tình không dùng AI.** Identity, encryption, phân quyền, gọi provider API đều là code thường, có test, deterministic. AI nằm đúng ở chỗ nó hơn code thường, và không nằm ở chỗ nào khác.